

Google Play Store Launch Readiness Audit: Invoice Hammer

This audit outlines the technical configurations, store credentials, signing keys, and store listing steps required to publish the Android version of **Invoice Hammer** on the Google Play Store.

1. Technical Configuration & Package Renaming

Package Name / Application ID Check

The package name (Application ID) is the unique identifier of your app on the Google Play Store and **cannot** be changed after publishing.

- **Current Configuration:** `com.fordham.toolbelt` (configured in `AndroidManifest.xml` and `app/build.gradle.kts`).
- **Action Required:** Change this to a professional package name (e.g. `com.invoicehammer` or `com.invoicehammer.app`).
- **Steps to rename:**
 1. Update `applicationId = "com.invoicehammer"` and `namespace = "com.invoicehammer"` in `app/build.gradle.kts`.
 2. Change authorities in `AndroidManifest.xml` for the `FileProvider` (`com.invoicehammer.provider`).
 3. Update any packages in `proguard-rules.pro` (change `com.fordham.toolbelt` to `com.invoicehammer`).

Target SDK Version

- **Current SDKs:** `compileSdk = 35`, `targetSdk = 35`, and `minSdk = 26` (Android 8.0).
- **Compliance:** This fully satisfies Google Play's target API level policy (which mandates new app submissions to target at least API 34 / Android 14).

2. Production Signing & Key Management

Google Play Console requires **Play App Signing** by default. In this flow:

1. Google manages and secures your app's **App Signing Key**.
2. You generate a local **Upload Key** in a keystore.
3. You sign your release build (.aab) with your **Upload Key**.
4. Google validates the upload signature and re-signs it with the App Signing Key before distribution.

Step 1: Create a Production Upload Keystore

Generate a fresh keystore specifically for production uploads (do not use the beta keystore or passwords for production):

```
keytool -genkeypair -v -keystore keystore/invoicehammer-upload.jks -alias upload -keyalg RSA -keysize 2048 -validity 10000
```

[!WARNING]

Keep invoicehammer-upload.jks extremely secure. If you lose this keystore, you will not be able to upload updates to your published app without contacting Google Play Developer Support to reset your upload key.

Step 2: Configure local.properties

Add the signing configurations to your local.properties (never commit these credentials to a public git repository):

```
release.storeFile=keystore/invoicehammer-upload.jks
release.storePassword=YOUR_PRODUCTION_STORE_PASSWORD
release.keyAlias=upload
release.keyPassword=YOUR_PRODUCTION_KEY_PASSWORD
```

3. Firebase Console & Google Sign-In Setup

Because Invoice Hammer uses Firebase Auth, Google Sign-In, and Google Drive Backups, **every signing fingerprint must be registered in the Firebase and Google Developer consoles**. Otherwise, Google Sign-In will fail with **Error Code 10**.

Register fingerprints for:

1. **Local Debug Keystore:** (Already registered in google-services.json).
2. **Local Upload Keystore:** The SHA-1/SHA-256 fingerprints of your newly created invoicehammer-upload.jks.
3. **Google Play App Signing Key:** After uploading your first AAB to the Play Console, locate the SHA-1 under:
 - **Play Console** -> **Setup** -> **App integrity** -> **App signing key certificate**.
 - Copy the SHA-1 and SHA-256 fingerprints.

Update Firebase:

1. Open the Firebase Console for project toolbelt-3812f381.
2. Go to **Project Settings** -> **Your Apps** -> **Android app**.
3. Under **SHA certificate fingerprints**, click **Add fingerprint** and paste the fingerprints.
4. Download the newly updated google-services.json and overwrite google-services.json.

4. Production API Keys & Environment Secrets

Ensure your local.properties points to production environments instead of sandbox or test instances:

| Property | Environment | Detail |

| :--- | :--- | :--- |

| supabase.url / supabase.anon.key | **Production** | Your production Supabase project keys. |

| stripe.publishable.key | **Live** | Set this to Stripe live publishable keys (pk_live_...).

| stripe.payment.backend.url | **Production** | Points to production Supabase Stripe Edge Function. |

| foreman.gemini.backend.url | **Production** | Points to production Supabase Gemini Agent Edge Function. |

| powerpay.env | production | Set to production for live Stellar PowerPay USDC payments. |

5. Store Listing & Compliance Requirements

Google Play requires a set of listing assets and legal materials:

Legal Compliance

- **Privacy Policy URL:** A hosted privacy policy URL is **strictly required** by Google Play for apps accessing user audio/microphones (RECORD_AUDIO) and cameras (CAMERA). You must display this URL inside the app (e.g. in Settings) and provide it in the Play Console listing.
- **Data Safety Form:** In the Play Console, you must complete the Data Safety questionnaire declaring:
 - Collection of contact info (if users sign up).
 - Audio files (captured voice fragments).
 - Images (receipt OCR photo scans).
 - Financial info (invoice amounts, payment methods).

Design and Visual Assets

- **App Icon:** High-resolution 512x512 PNG (replace any placeholder launcher icons in res/mipmap).
- **Feature Graphic:** 1024x500 PNG.
- **Store Screenshots:** Phone (at least 4 screenshots), 7" tablet, and 10" tablet layouts.

6. How to Build & Verify the Release Package

To generate the final production bundle, run:

```
# Clean build cache and compile the production Android App Bundle (AAB)
.\gradlew clean :androidApp:bundleRelease --no-daemon
```

- **Output Path:** app/build/outputs/bundle/release/app-release.aab
- Upload this .aab file to your **Internal Testing**, **Closed Testing**, or **Production** track inside the Google Play Developer Console.

7. Google Play Console Account & Merchant Setup

If you are offering in-app purchases (like the Hammer Credit Packs and Pro Subscriptions), you must configure developer account-level integrations:

Play Store Merchant Profile

- **Action Required:** Link a Merchant Profile to your Developer Console to accept payments. Go to **Settings** -> **Developer Account** -> **Merchant Account** and complete your business registration.
- **Tax & Payouts:** Set up your bank account and tax forms to receive payouts from Google Play transactions.

Register In-App Products & Subscriptions

You must create products inside the Play Console that match the product IDs defined in the app code. If these IDs do not match exactly, purchases will fail:

1. **Consumables (Hammer Credits):**
 - hammer_credit_pack_50 (50 credits)
 - hammer_credit_pack_150 (150 credits)
 - hammer_credit_pack_400 (350 credits - matches PurchasableProduct.HammerCreditPack400)

2. ****Subscriptions (Pro Tier):****

- invoice_hammer_pro_monthly (Pro Monthly)
- invoice_hammer_pro_yearly (Pro Yearly)

8. Play Store Closed Testing Policy (Organization Account Exemption)

NOTE:

Since you are using an ****Organization Account**** (registered business/corporate developer account), your project is ****exempt**** from the new developer 20-tester/14-day closed testing rule. You can proceed directly to publishing on the Open Testing or Production tracks without running the 14-day closed beta.

9. Preparing for Google App Review

When you submit the app for review, Google policy reviewers must be able to log in and test all premium/auth-restricted features.

- ****Action Required:**** In the Play Console under ****App Content**** -> ****App Access****, select ****All or some functionality is restricted**** and provide:
 1. A ****test account username and password**** (create a dedicated test email/password user in Firebase Auth).
 2. Brief instructions detailing how to use the Gemini Voice Agent or scan a dummy receipt.